

Trace Context Beyond HTTP

Environment variables as OpenTelemetry propagation carriers



Robert Pająk

github.com/pellared



Alan Clucas

github.com/Joibel

Where is this?

Hlubočepské plotny

Prague

A beautiful spot for rock climbing



SPLUNK

Robert Pająk

@pellared

github.com/pellared

OPEN SOURCE

OpenTelemetry maintainer



PIPEKIT

Alan Clucas

@Joibe1

github.com/Joibe1

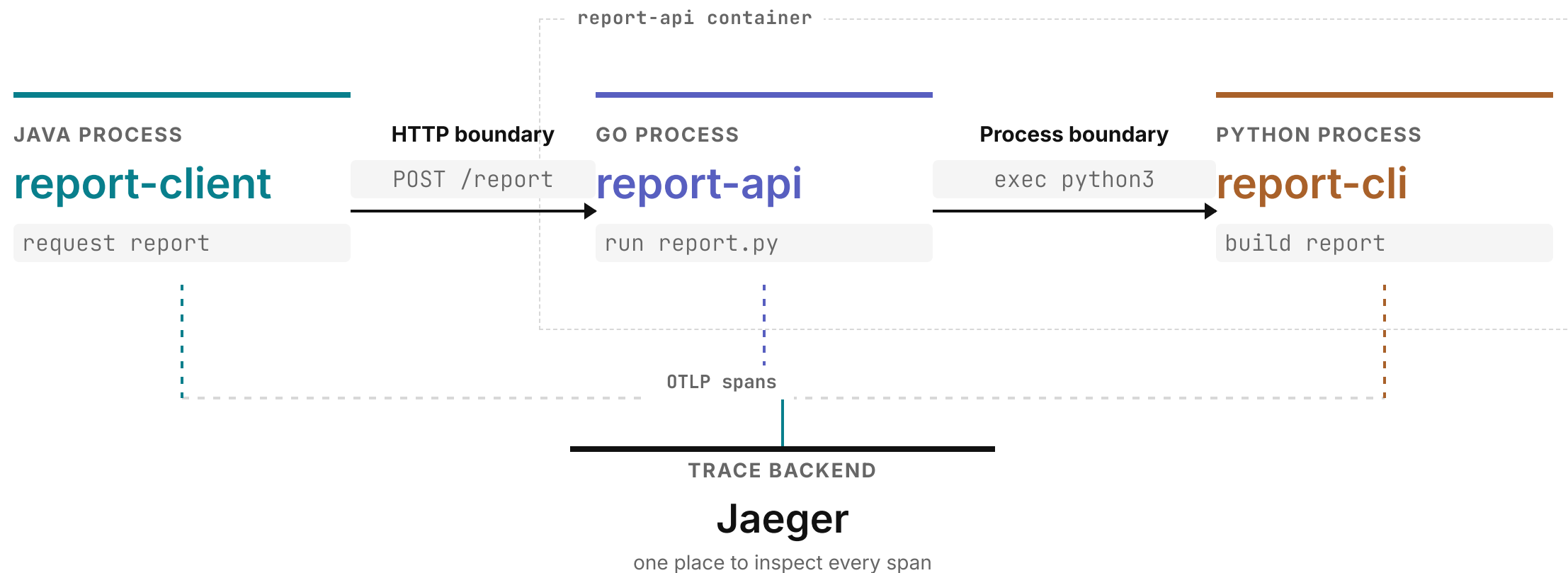
OPEN SOURCE

Argo Workflows lead



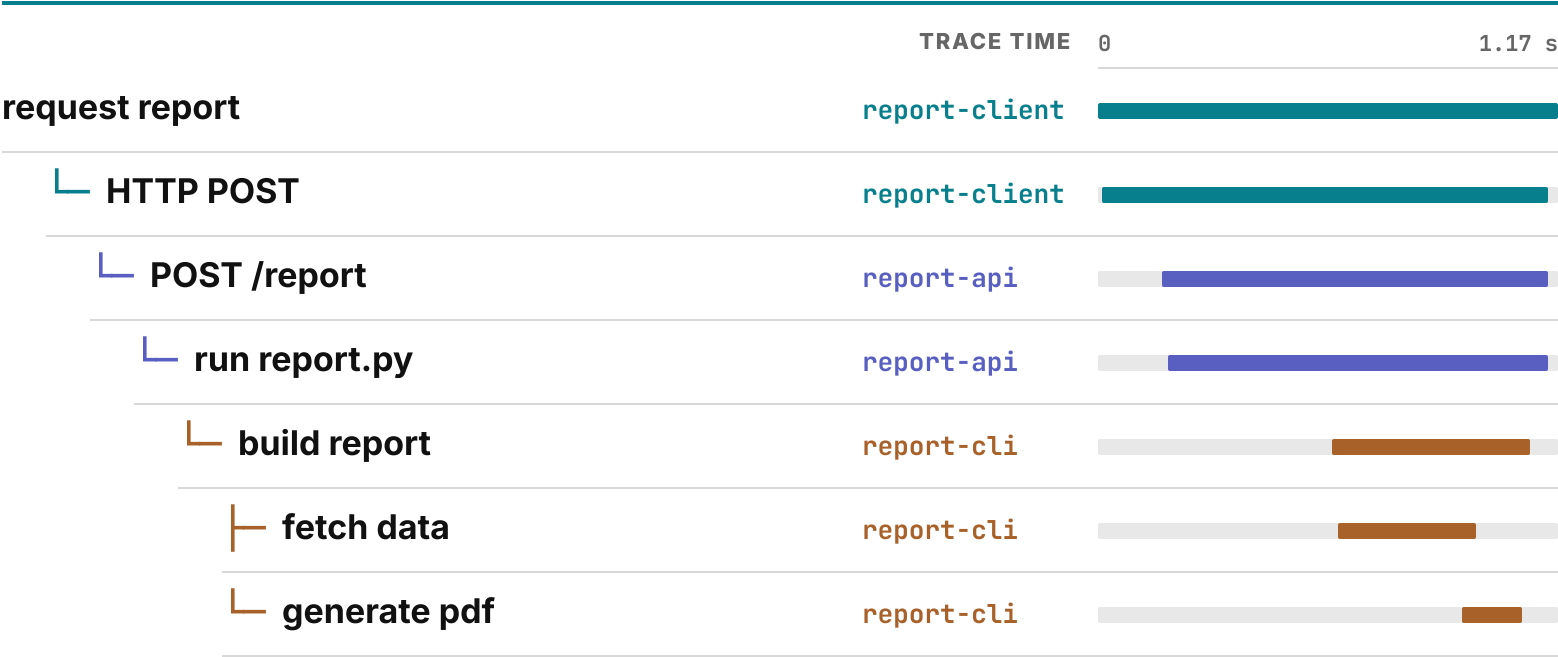
DEMO 1

HTTP service starts a CLI



Trace, spans, and parentage

ONE TRACE d2264e9b...4308af96



Trace

The complete causal story

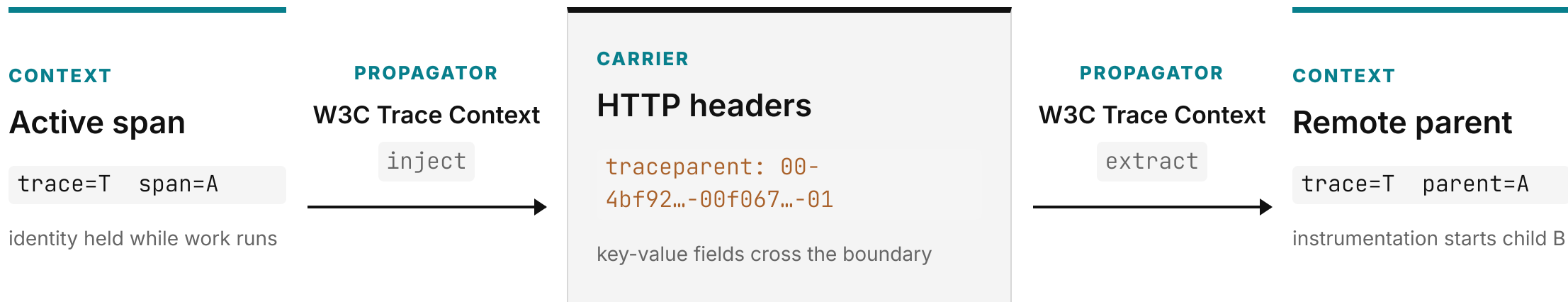
Span

One timed operation

Parent

The work that caused the next span

Context propagation over HTTP

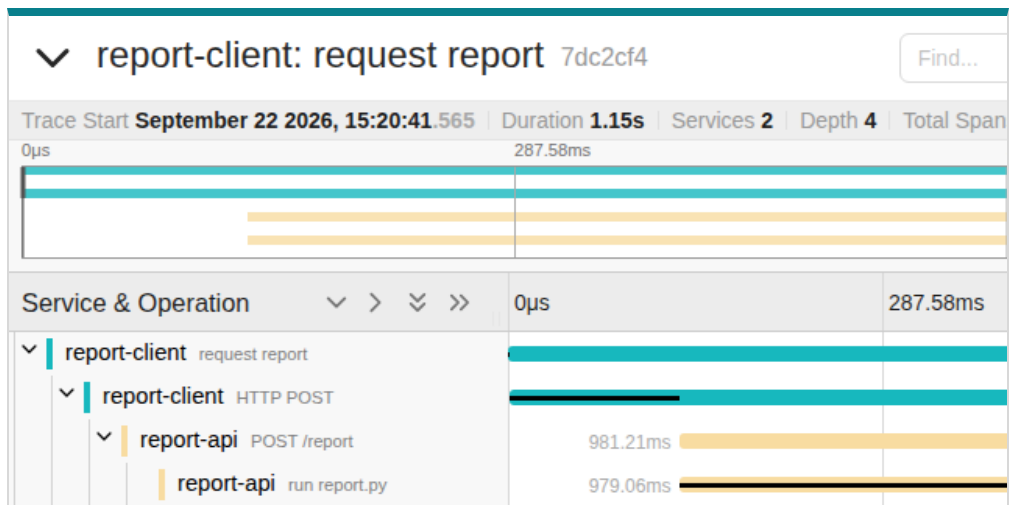


MISSING HANDOFF

HTTP headers stop at the process boundary

Request trace

7dc2cf4...

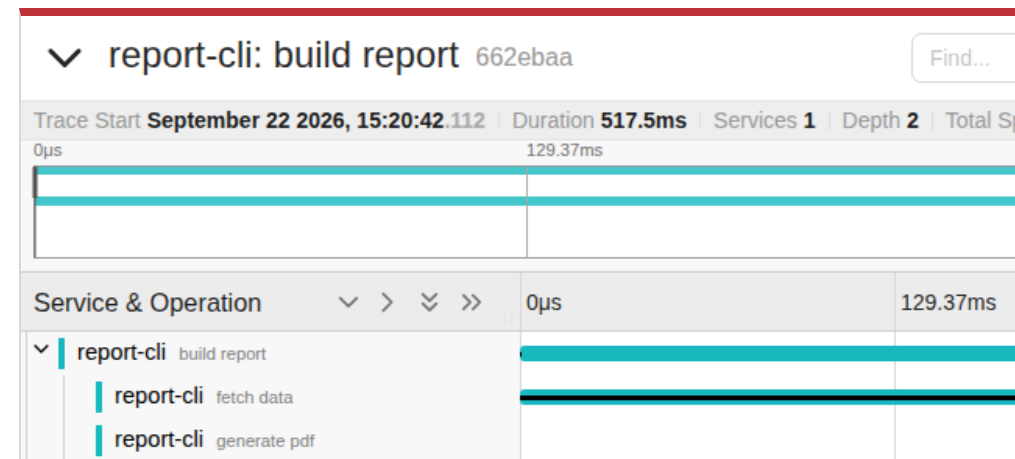


Stops at `run report.py`

4 spans · Java → Go

Python root trace

662ebaa...



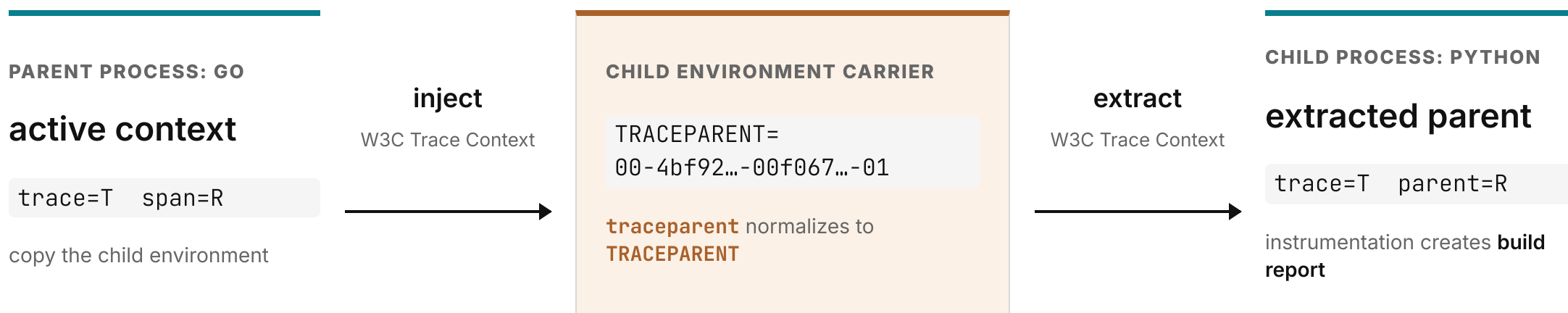
Starts at `build report`

3 spans · Python only

≠

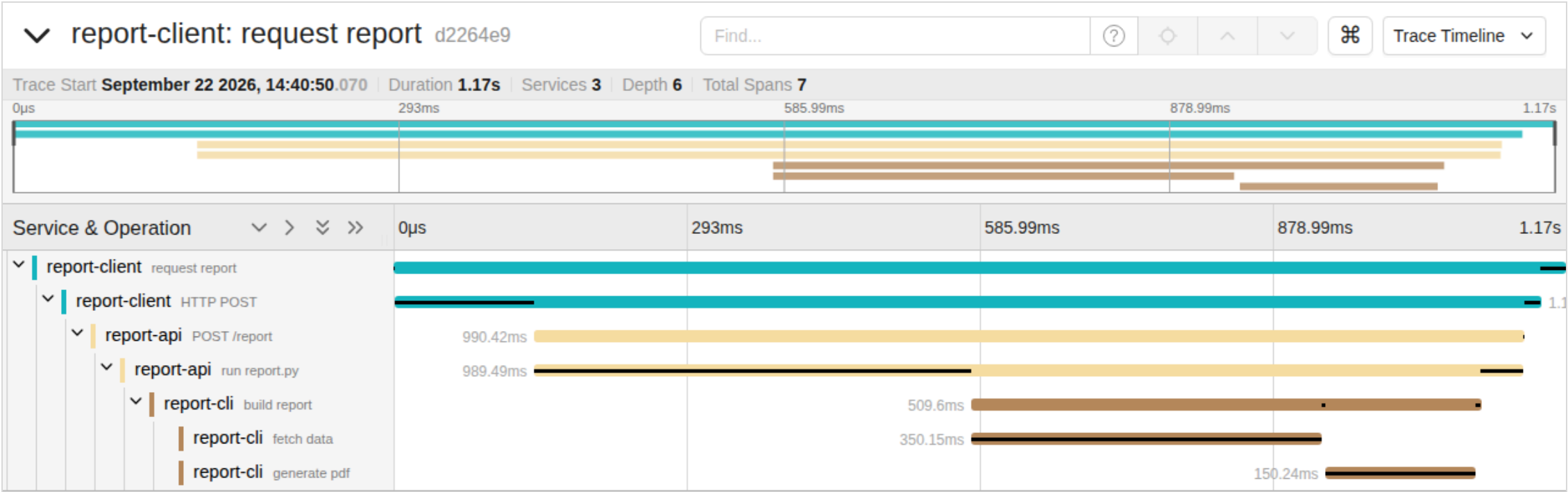
DIFFERENT
TRACE IDS

The child environment carries the handoff



The propagator and format stay the same. The carrier matches the boundary.

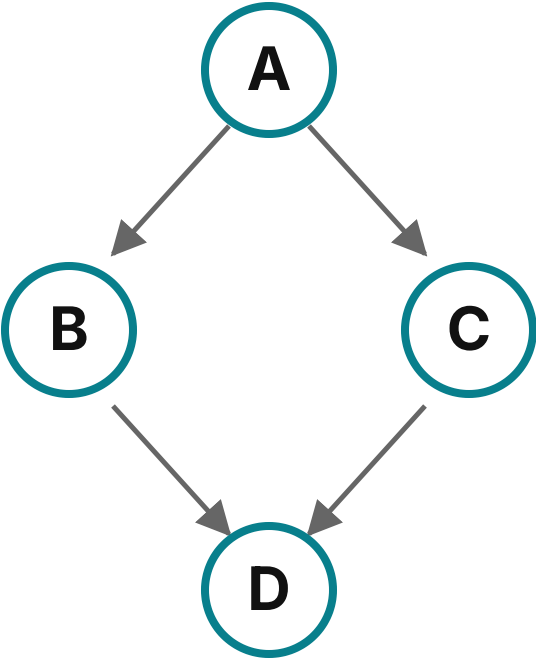
The CLI joins the original trace



1 trace 7 spans across 3 services

Environment carried parentage. Instrumentation created the spans.

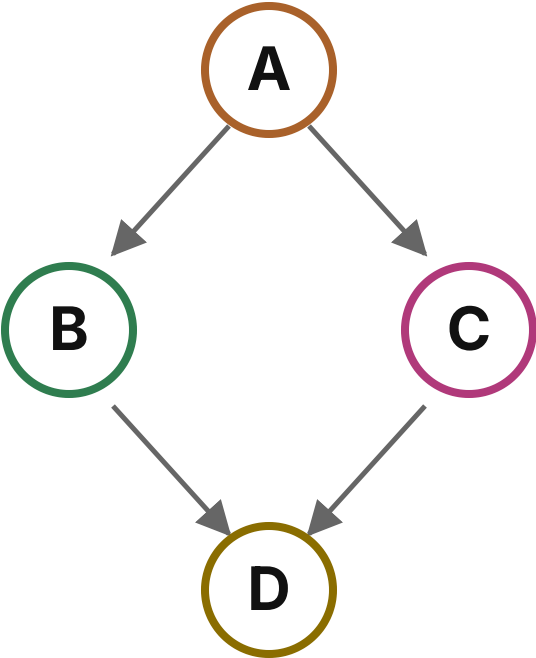
Tracing a workflow



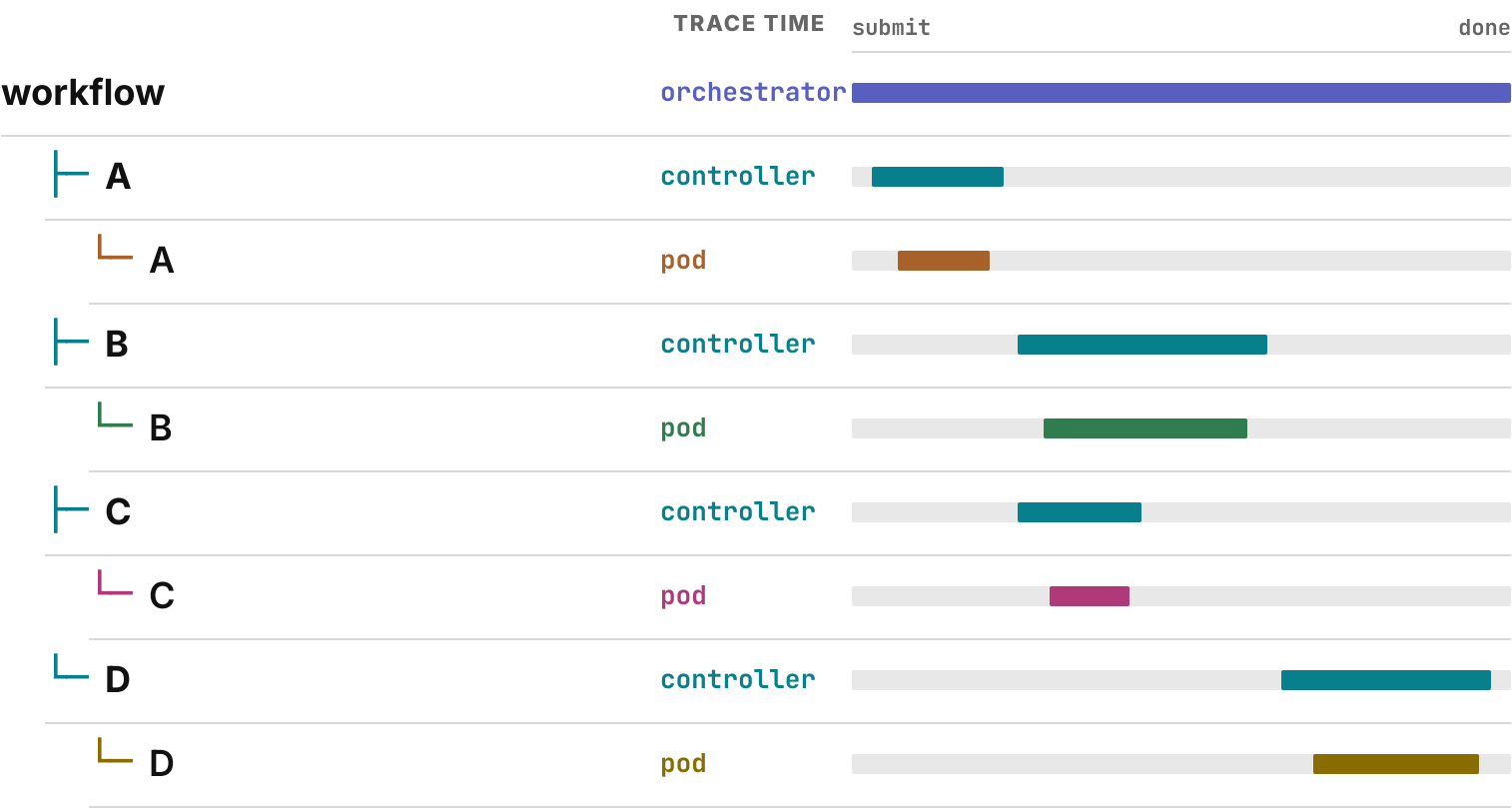
workflow

		TRACE TIME	submit	done
		orchestrator		
└ A	step			
└ B	step			
└ C	step			
└ D	step			

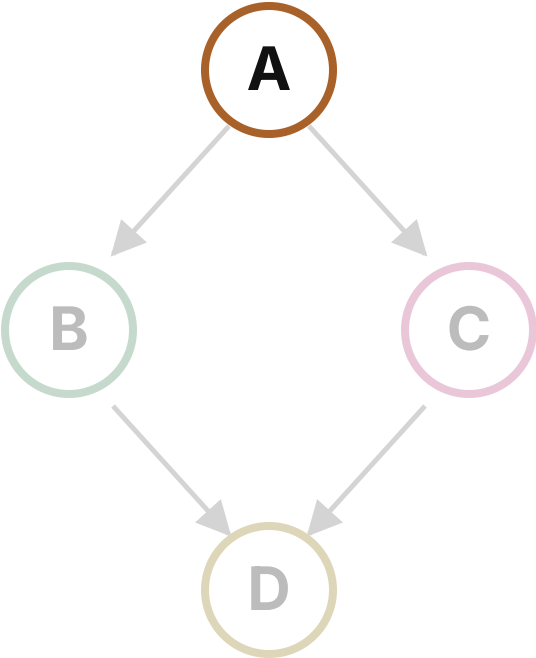
Inside the pods



workflow



What the workload did



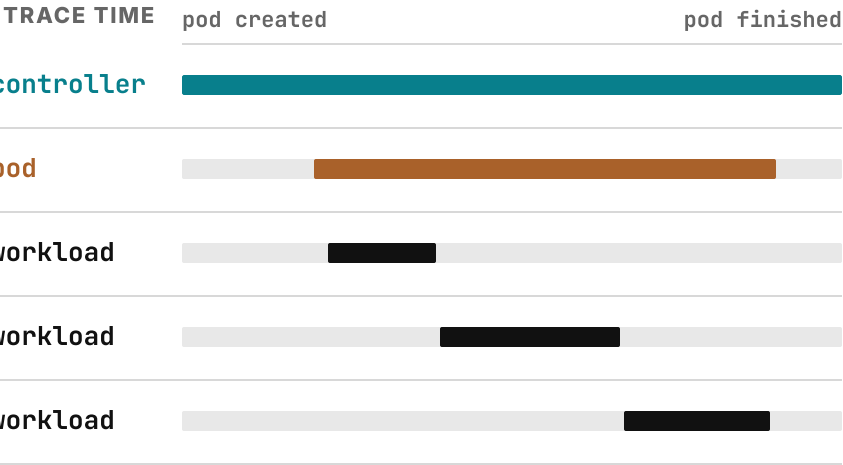
A

└ A

└ observability

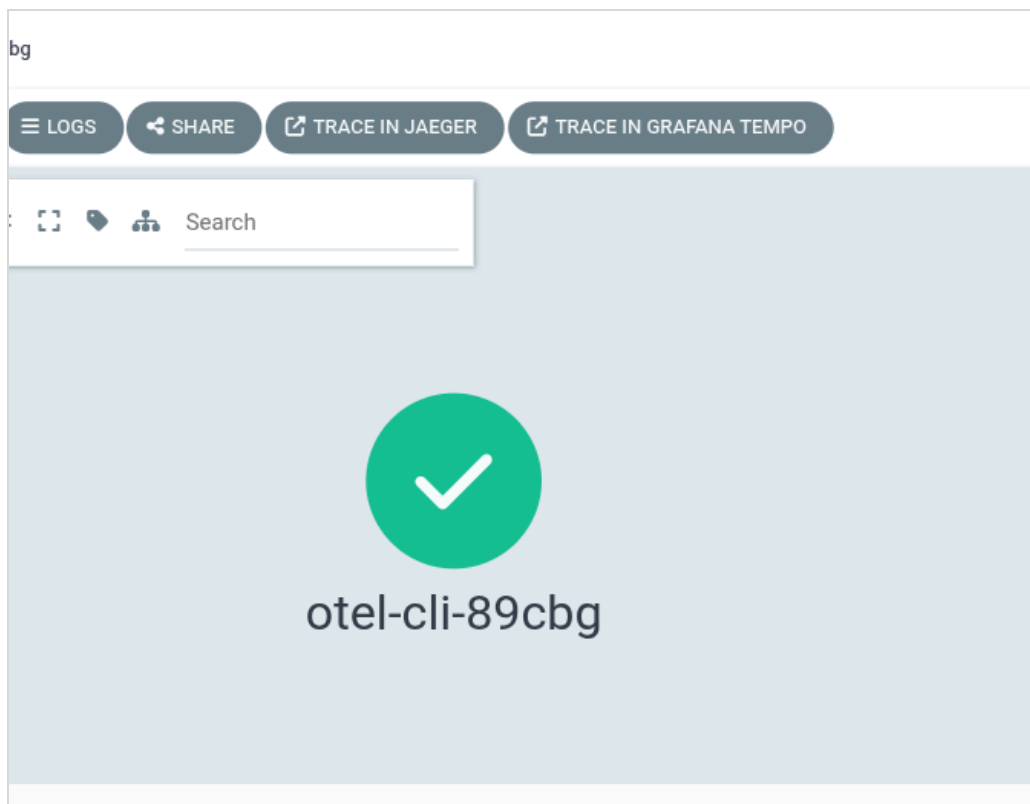
└ summit

└ prague



DEMO 2

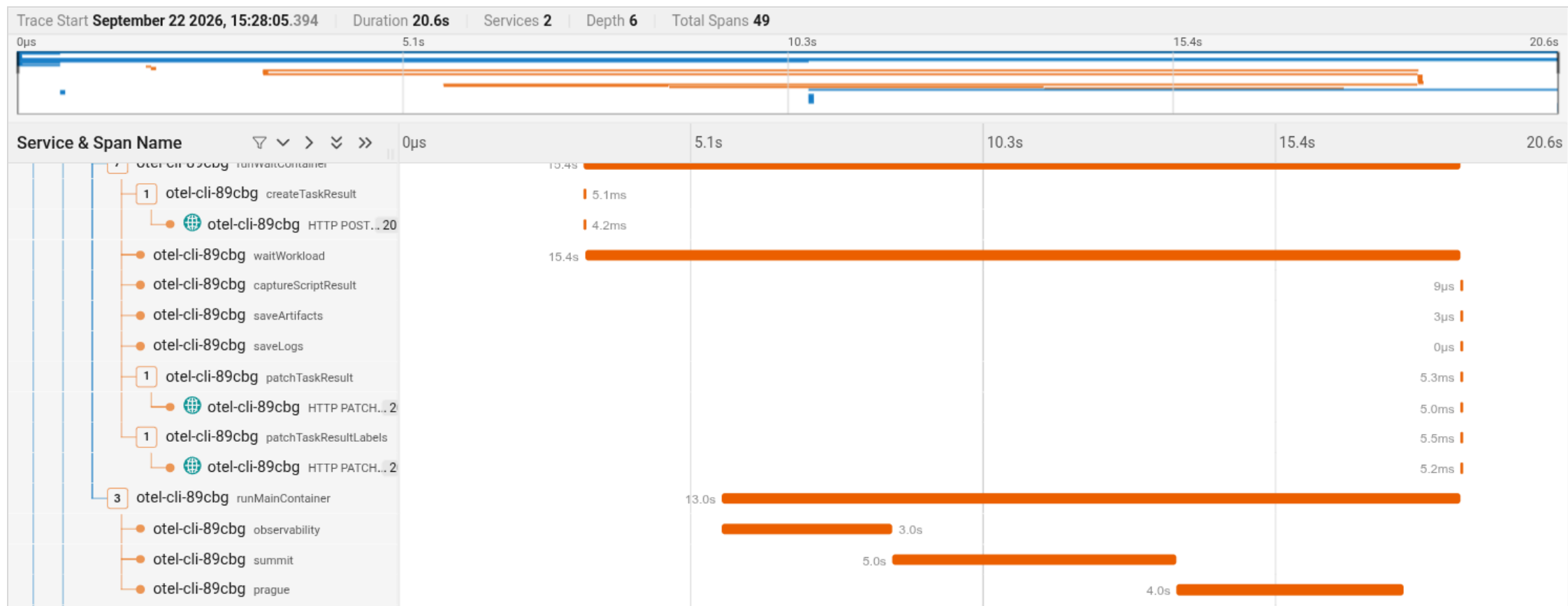
One step, three spans



```
echo "TRACEPARENT: ${TRACEPARENT}"  
/otel-cli exec --name observability -- sleep 3  
/otel-cli exec --name summit        -- sleep 5  
/otel-cli exec --name prague        -- sleep 4
```

DEMO 2

The workload joins the trace



1 trace 49 spans across 2 services

DEMO 2

Two injections

argoexec → workload

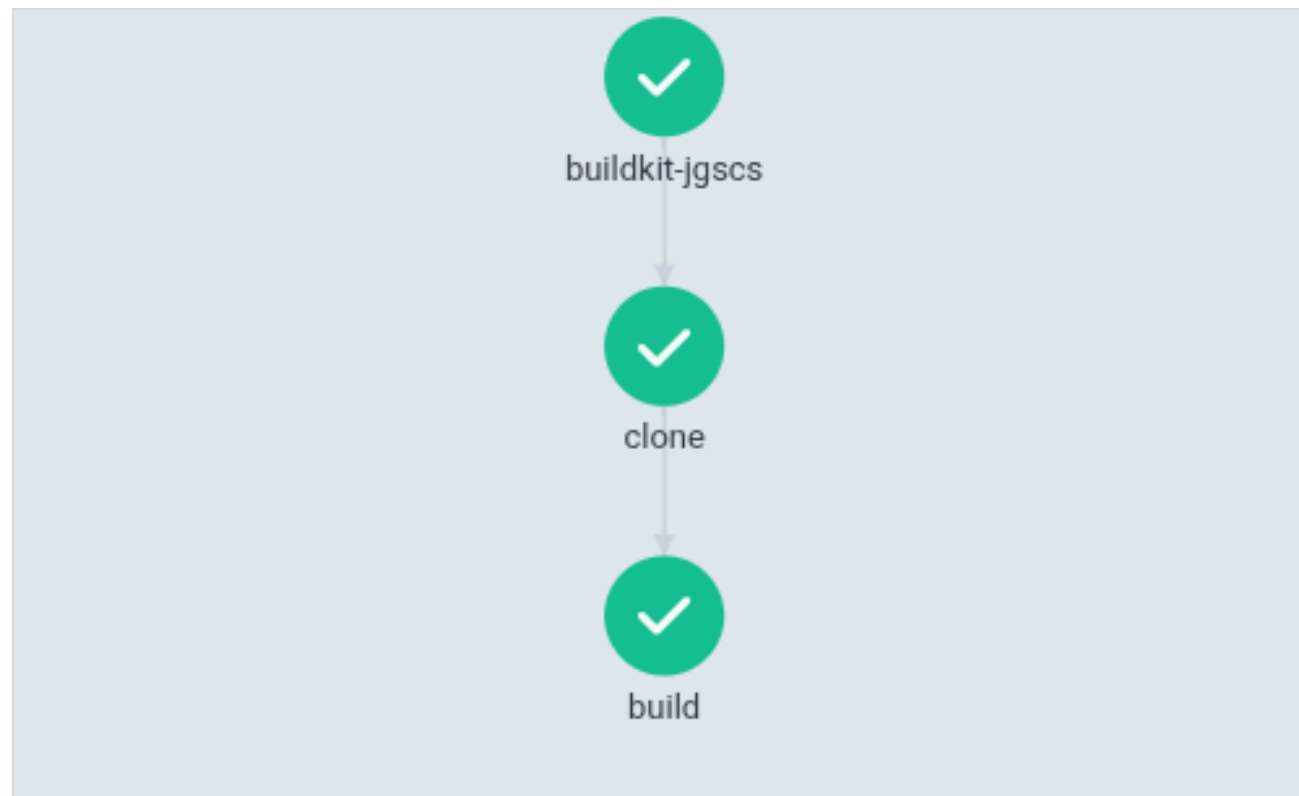
```
ctx, span := tracer.StartRunMainContainer(ctx, ...)
defer span.End()
env := os.Environ()
carrier := &envcar.Carrier{
    SetEnvFunc: func(key, value string) {
        env = setEnvVar(env, key, value)
    },
}
propagation.TraceContext{}.Inject(ctx, carrier)
cmd := exec.CommandContext(ctx, name, args...)
cmd.Env = env // the child's env, not argoexec's
```

controller → pod

```
carrier := &envcar.Carrier{
    SetEnvFunc: func(key, value string) {
        envVars = append(envVars,
            apiv1.EnvVar{Name: key, Value: value})
    },
}
propagation.TraceContext{}.Inject(ctx, carrier)
// TRACEPARENT=00-⟨trace>-⟨span>-01
```

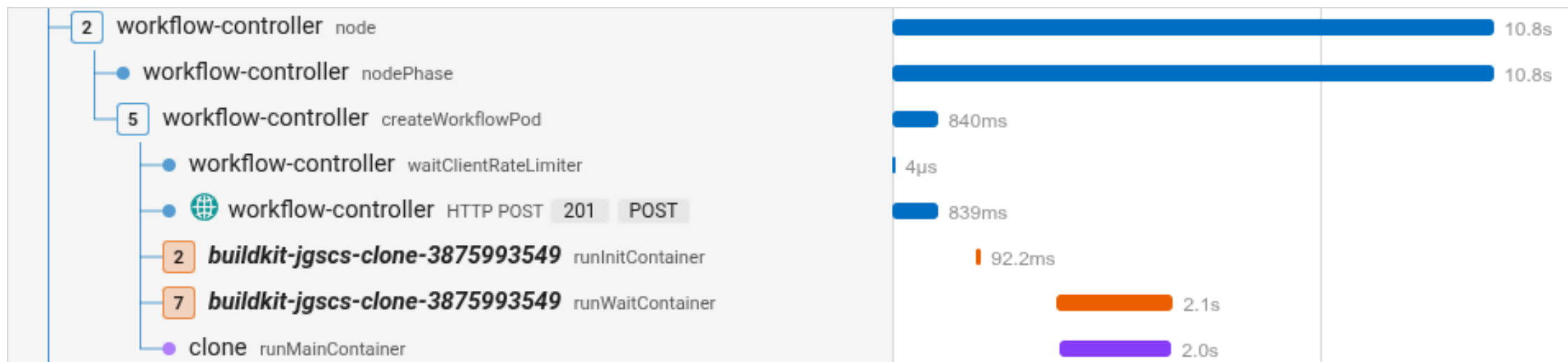
DEMO 3

Clone and build in Argo



DEMO 3

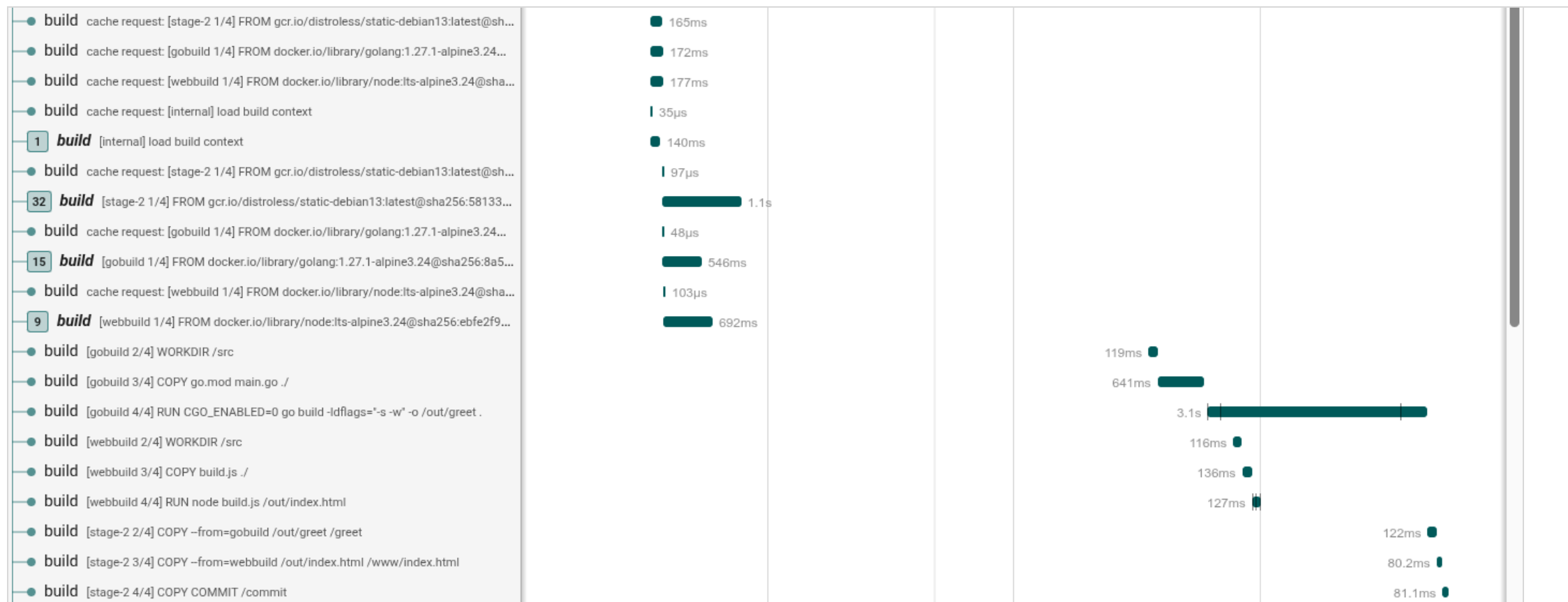
git says nothing



```
git clone --depth 1 https://github.com/Joibel/otel-deploy /src
```

DEMO 3

BuildKit reveals the build graph



Same carrier ~250 spans inside one step

DEMO 4

Batch lineage workflow

PLATFORM TEAM

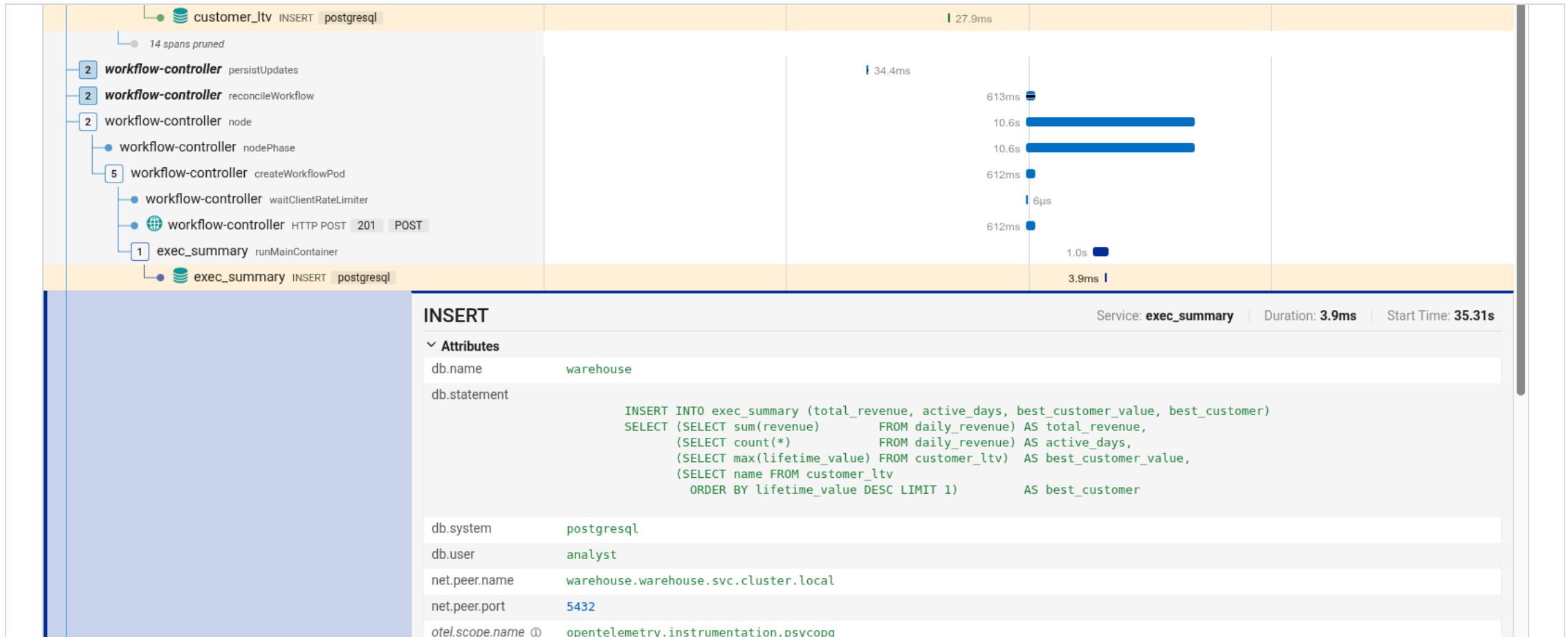
wants data lineage

DATA SCIENTISTS

own the code



DEMO 4 Every query is a span



No tracing code psycopg, auto-instrumented

DEMO 4

The lineage, from the trace

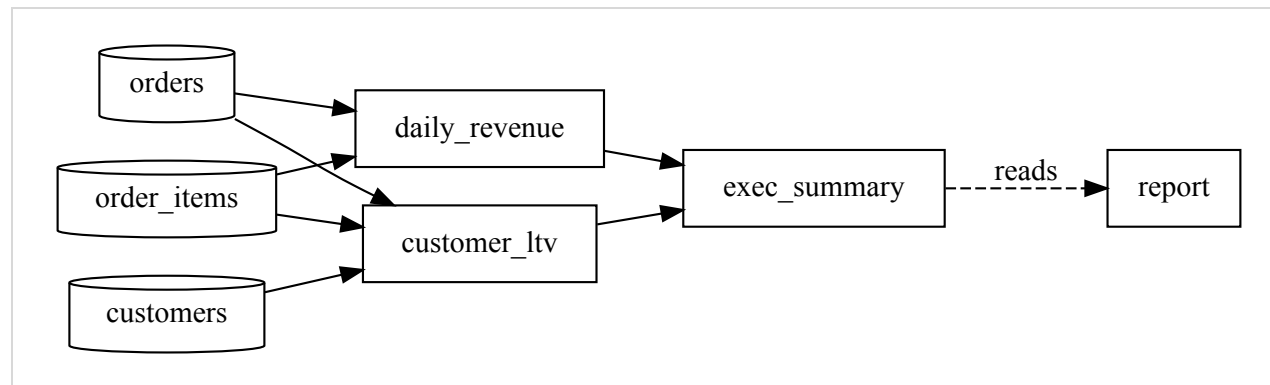
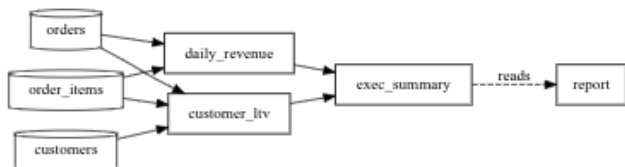
lineage-report

artifact:s3:minio.minio.svc.cluster.local:9000:my-bucket:datasci-lineage-bmw59/datasci-lineage-bmw59-lineage-3642270036/lineage.html

Data lineage

trace 479f95146b2c066fa838befdcde8c62

Derived from the SQL captured in this workflow's trace spans, not from the pipeline source. Cylinders are source tables, boxes are tables the pipeline built. A dashed edge is a stage that read a table without writing anything. Edges carry the stage name only where it differs from the table it produced; the table below has the full stage-by-stage breakdown.



DEMO 4

Python reads the carrier too

```
from opentelemetry.propagators._envcarrier import EnvironmentGetter
ctx = extract(os.environ, getter=EnvironmentGetter())
context.attach(ctx) # TRACEPARENT becomes the parent
runpy.run_path(script, run_name="__main__")
```

```
command: [python, -m, tracectx] # wraps the untouched pipeline
```

ADOPTION

Tools already use TRACEPARENT

Argo Workflows

injects context into pods and commands

Docker BuildKit

reads and forwards process context

Claude Code

links headless sessions and subprocesses

otel-cli

wraps a command in a span

Thoth

instruments shells and GitHub Actions

Jenkins OpenTelemetry plugin

exposes context to build steps

Shared field names let instrumented tools join without another custom adapter.

One carrier, several formats

W3C TRACE CONTEXT

```
TRACEPARENT=00-4bf92...-00f067...-01
```

trace and parent identity

W3C BAGGAGE

```
BAGGAGE=build.id=42,repository.name=example
```

application context for
downstream work

B3

```
X_B3_TRACEID=463ac35c9f6413ad
```

normalized from **x-b3-traceid**

Carrier: opaque strings | **Propagator:** field names, format, and validation

RELEASE CANDIDATE

Feedback before stabilization

EARLIEST PLANNED STABILIZATION

2 Nov 2026

After at least 14 days without a new related issue, and only if no blocker remains.

Normalization, portability, concurrency, security



Read the proposal
Report what breaks

[opentelemetry.io/blog/2026/
environment-variable-context-
propagation](https://opentelemetry.io/blog/2026/environment-variable-context-propagation)

KEEP EXPLORING

Slides and demo material

Presentation

Demo source

github.com/pellared/otel-env-car-talk



Open the repository

THANK YOU

Questions?

Robert Pająk

github.com/pellared

Alan Clucas

github.com/Joibel

CNCF Slack

[#otel-cicd](#)

OpenTelemetry CI/CD SIG

Open community meeting